Presentation Topics: Answers

the backend and security ideas behind each question

Dinesh Rawat

Instructor · Shorter Loop × Snapied

> These are worked answers to the five presentation topics. The point of every one is the same: to make you think like a backend developer, someone who assumes things will be attacked and builds so they cannot break. Read your own topic closely, but read all five, because they connect into one lesson. Where a real incident is involved, the facts here are from news reports; check them yourself before you present.

1 The CBSE portal, and how marks could be changed

Himanshu's topic. What actually happened, from the news reports. In 2026 a 19-year-old named Nisarga Adhikary, who had just finished class 12, looked at CBSE's On-Screen Marking (OSM) portal, the site where teachers evaluate scanned answer sheets online. He found a set of basic security holes and reported them. CBSE first denied any breach, saying it was only a test site, then later acknowledged vulnerabilities in the provider's portal and ordered an audit. He did not do it for gain, he reported it, which is exactly what an ethical hacker does.

the flaws, and what each one teaches

The flaw	The backend lesson
A master password was written into the source code, visible in the browser.	Never put secrets in code that reaches the browser. Anyone can read it with Ctrl+F.
The OTP check could be bypassed.	A security check the client can skip is not a security check.
Password reset did not ask for the old password. Any user ID plus any gibberish reset it.	The server must verify every step. Never trust that the client did the checking.
An examiner's identity could be edited through browser storage, letting you impersonate a teacher.	Never trust data from the browser. The server decides who you are, not the client.
Internal dashboards were open with no login.	Every private route needs an access-control check, on the server, every time.

The one sentence answer. The marks were changeable because the system trusted the browser. Passwords, identities and checks all lived where the user could see and edit them. His own words: "These aren't advanced defences. They're the basics." A backend developer's first rule is the opposite of what CBSE did: never trust the client, verify everything on the server.

2 How 10,000+ records get created in minutes

Deepesh's topic, and it happened in Shorter Loop. A human types slowly. To create 10,000 records by hand would take days. So it was not done by hand. It was done by a script, a small program that repeats an action in a loop, as fast as the server will allow.

> **Script or bot**
> A program that does a repetitive task automatically. A few lines can send the same "create record" request thousands of times in a loop. What takes a person all day takes a script a few seconds. This is not advanced, it is a beginner-level loop pointed at an endpoint that was not protected.

why a site lets this happen, and how you stop it

What was missing	The fix a backend developer adds
No rate limit. The server accepted requests as fast as they came.	Rate limiting: cap how many requests one user or IP can make per minute.
No bot check on the form.	A CAPTCHA or similar on public create-endpoints.
The endpoint was open, no login needed to create records.	Require authentication, so anonymous scripts cannot post at all.
No validation catching obviously fake or duplicate data.	Server-side validation and duplicate checks that reject junk.

The one sentence answer. 10,000 records appeared because a script hit an endpoint that had no rate limit, no bot check and no login. The lesson: assume any open endpoint will be hit by a machine, not a polite human, and put limits in before it happens, not after.

3 What information is kept in the UI

Divya Bora's topic. The UI is the HTML, CSS and JavaScript that runs in the browser. The single most important fact about it: everything in it is visible to everyone. Anyone can right-click, choose "view source" or open developer tools, and read every line of your HTML, CSS and JavaScript. Nothing there is hidden.

What lives in the UI	So remember
The page structure and text (HTML)	readable by anyone, always
The styling (CSS)	readable, harmless, fine to be public
The logic that runs in the browser (JavaScript)	fully readable. Never hide a secret in it.
Values stored in the browser (local storage, cookies)	readable AND editable by the user

The one sentence answer. The UI holds only what you are willing to show the whole world, because the whole world can read it. This is exactly the CBSE mistake from topic 1: a master password sat in the browser code, and browser storage decided who you were. Rule: never put a secret, a password, or a trust decision in the UI. The browser is the user's territory, not yours.

4 Who are ethical hackers

Gaurav and Ayush's topic. An ethical hacker uses the same skills as an attacker, but with permission, to find security holes so they can be fixed before a real attacker uses them. Same knowledge, opposite intent, and crucially, done legally and reported responsibly.

Ethical hacker	Malicious hacker
has permission, or reports responsibly	no permission, hides the act
finds a hole and tells the owner to fix it	uses the hole for gain or damage
makes systems safer	makes victims
paid by companies, bug bounties, security jobs	faces jail

Nisarga from topic 1 is the live example. He found serious flaws and reported them to the authorities and CERT-In instead of changing marks for money. That single choice, report it rather than abuse it, is the whole line between the two columns. Companies pay ethical hackers, run bug bounty programs, and hire security teams precisely because they would rather a friendly researcher find the hole first.

The one sentence answer. Ethical hackers are the people who find the holes before the criminals do, with permission and in the open, and it is a real, paid career. The skills are the same as an attacker's. The intent and the legality are what make them ethical.

5 The movie: Pritam Pedro

Unishka and Divyani's topic. The film is Pritam Pedro, Rajkumar Hirani's 2026 cybercrime thriller on JioHotstar. It pairs two opposite cops: Pedro, an old-school investigator who trusts traditional methods, and Pritam, a young tech-savvy officer who works through modern technology and code. Together they solve cybercrimes, starting with a mystery around an abandoned ATM on a beach and building to a kidnapping case. The Hindu described it as a story where "old-school muscle beautifully collides with modern computer code." Watch it, and take notes on what it shows about how technology and crime meet, and about the backend developer's way of thinking.

watch for these, and note them

- The two mindsets. Pedro trusts instinct, Pritam trusts the system and the code. A backend developer

needs both: the discipline to verify everything, and the instinct to sense when something is wrong. Notice where each approach wins.

- Where does trust break? A cybercrime happens because someone trusted something they should not have, a system, a person, a piece of data. This is the exact lesson from the CBSE topic. Note every point where misplaced trust opens a door.
- What is exposed, and what is hidden? Cybercrime turns on information: what an attacker can see, reach, or fake. Connect it to what we learned about the UI exposing everything, and identities being faked through the browser.
- How is the attacker caught? The tech-savvy side of the investigation is backend thinking in action: following data, spotting what does not add up, understanding how the system really works underneath.

the coding side: what a clicked link actually reveals

Your practical task is the location question, so present the honest version of it, because most people get it wrong. When someone clicks a link you sent, you do not get their exact location. You get their public IP address, and an IP maps only to a rough area: a city or the internet provider's region, and it is often wrong for mobile data, VPNs and company networks.

What people believe	What actually happens
a clicked link reveals exact location	it reveals the public IP, which is city or ISP level at best
you can read their local address (192.168.x.x)	you cannot. it stays inside their router and never reaches any server

> **Why the local IP never arrives**
>
> The person's device has a private address like `192.168.1.7` inside their home network. On the way out, their router swaps it for the one public IP the whole house shares. This is called NAT. Your server only ever receives that public IP. The private one never leaves their network. So no server, in any language, can read a visitor's local IP from a request.

So how do the stories of "they found my exact location from a link" happen? Almost always one of these, and none of them is the link by itself:

How exact location really leaks	What it needs
the page asks "Allow location?" and the person taps Allow	the person's consent. the browser's GPS prompt cannot be skipped
the person uploaded a photo or posted with location on	the person handing the data over themselves
malware or a hacked account or phone	the device already compromised. a different, illegal thing
police mapping an IP to a real address	a legal request to the internet provider. not something a normal person can do

The one sentence answer for the location task. A link on its own leaks only the public IP, which is a rough area, never a street address and never the local IP. Exact location requires the person's consent, their own leaked data, or something already compromised. The gap between what people fear a link reveals and what it actually reveals is the real lesson, and it is pure backend thinking: the server only sees what the request carries, and everything precise stays behind a wall the user controls.

What to bring back. Do not just retell the plot. Bring three moments from the series that connect to something we have learned: misplaced trust, exposed information, scale, or how the crime was traced through the system. Show the batch how a cybercrime thriller is really a lesson in how systems break, and how a backend developer thinks to keep them from breaking. That is watching it like a backend developer.

6 The thread through all five

One idea, five topics

Marks were changed because the browser was trusted. Records were flooded because an endpoint had no limits. The UI exposes everything because it runs on the user's machine. Ethical hackers are the people who find these gaps first. And the film is there to make the mindset stick. The single sentence under all of it: `a backend developer assumes attack, never trusts the client, and verifies everything on the server`. Present your own topic well, but show the batch how it connects to this.

Everything here ran live in class. If a part does not click, message me the file name and the line, and read it again tonight while it is fresh.